

M2_Practice

Danni Martin

2024-11-07

Functions for Generating Random Numbers

Distribution	Family	Package	Function
Uniform	Discrete	extraDistr	rdunif (n, min, max)
Binomial	Discrete	Base R	rbinom (n, size, prob)
Hypergeometric	Discrete	Base R	rhyper (nn, m, n, k)
Poisson	Discrete	Base R	rpois (n, lambda)
Uniform	Continuous	Base R	runif (n, min, max)
Normal	Continuous	Base R	rnorm (n, mean, sd)
Exponential	Continuous	Base R	rexp (n, rate)
Triangle	Continuous	extraDistr	rtriang (n, a, b, c)

Notes: size = number of trials Hypergeometric - similar to binomial but sampling without replacement. ie drawing cards from a deck. Poisson - events occurring within a specific period of time

Recall that the **binomial distribution** illustrates the probability of x successes from an experiment with n trials. We can use the distribution to generate random numbers by providing the p (probability of success) and n (number of trials) parameters. Similarly, the **uniform distribution** shows the probability of a random variable within a minimum and maximum limit. Hence, we can generate random numbers from the distribution by providing the minimum and the maximum. In general, we can generate random numbers that follow a variety of distributions by providing the required arguments to the particular random number generator needed.

Case 1: Nightmare Reads Simulates Demand

Nightmare Reads bookstore wishes to determine how many customers will buy their monster classics series. They plan to send 100 catalogs by mail to potential customers. Before they send

the catalogs, they decide to get an estimate on demand. Past data reveals that a customer will buy a book from the catalog with a probability of 0.70. Using R, they simulate the demand generated by their catalog.

```
# Create df for monster series books with price. Use rbinom to generate demand.
#Specify numbers needed (5), number of trials (100), and probability customer
#will buy book (0.7).
(MS<-data.frame(Books=c("Frankenstein","Dracula",
                        "Moby Dick",
                        "War Of The Worlds","Beowulf"),
                Price=c(9.5,5.78,6.34,5.67,2.45),
                Demand=rbinom(5,100,0.7)))
```

	Books	Price	Demand
1	Frankenstein	9.50	66
2	Dracula	5.78	70
3	Moby Dick	6.34	72
4	War Of The Worlds	5.67	73
5	Beowulf	2.45	67

Using Loops in R

```
# Create a for loop to simulate demand for EACH book.
Revenue<-c()

for (i in MS$Price) {
  Revenue<-c(Revenue,i*rbinom(1,100,0.7))
  print(Revenue)
}
```

```
[1] 722
[1] 722.00 398.82
[1] 722.00 398.82 462.82
[1] 722.00 398.82 462.82 368.55
[1] 722.00 398.82 462.82 368.55 181.30
```

Vectorization - Loops

```
# Can accomplish the same task by multiplying Demand by Price.
# This process is called "vectorization".
Revenue<-MS$Demand*MS$Price
Revenue
```

```
[1] 627.00 404.60 456.48 413.91 164.15
```

Using Conditionals in R

```
# Modify simulation- probability is 0.9 if the books are 'Dracula' or 'Frankenstein'.
demand<-c()

for (i in MS$Books){
  if (i=="Frankenstein" | i=="Dracula"){
    p=0.9
  }
  else {p=0.7}
  demand<-c(demand, rbinom(1,100,p))
}

print(demand)
```

```
[1] 87 89 67 67 67
```

Vectorization - Conditions

```
demand<-ifelse(MS$Books=="Frankenstein" | MS$Books=="Dracula",
               rbinom(length(MS$Books),100,0.9),
               rbinom(length(MS$Books),100,0.7))
demand
```

```
[1] 90 84 69 75 70
```

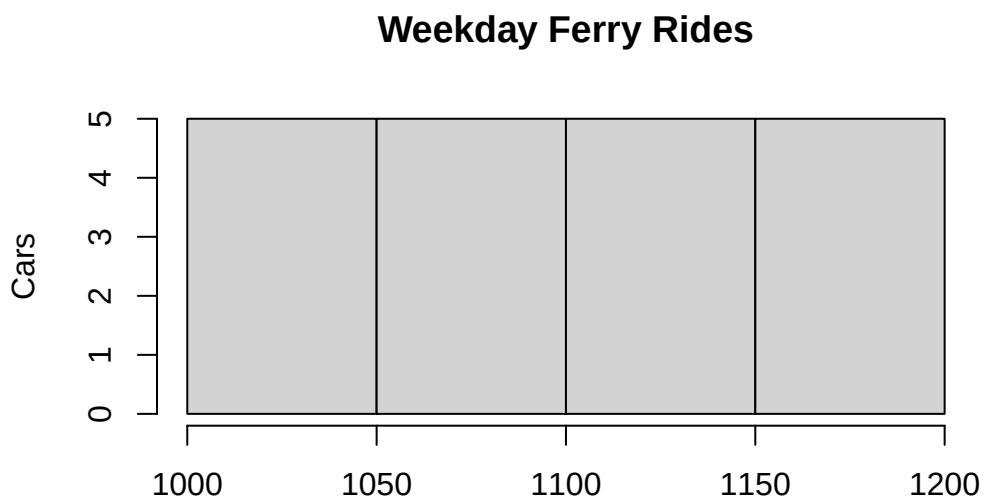
Case 2: VA Department of Transportation

Let's consider another case study in which we could apply a simulation. The VA ferry crossing the James River was first established in 1925. The ferry transports vehicles back and forth from Jamestown to Scotland in a 15-minute ride. The VA Department of Transportation wants you to simulate the daily demand for the ferry so that they can appropriately schedule staff and the correct number of ferries to run.

Assume that the VA Department of transportation shares four weeks of data with hopes of us being able to simulate the fifth week. Can we generate a convincing simulation of the data?

```
# Create vectors to capture data.
weekdays<-c(1105,1020,1163,1070,
            1128,1048,1066,1145,
            1189,1102,1183,1083,
            1175,1094,1003,1045,
            1101,1162,1095,1018)
weekends<- c(1459,1464,1408,1443,
            1580,1534,1512,1599)

# Visualize data with histogram
hist(weekdays, main="Weekday Ferry Rides", xlab="",
     ylab="Cars")
```



Ferry rides are uniformly distributed across weekdays.

```
# Using a for loop and condition, simulate data for week 5.
library(extraDistr)
set.seed(14)

sim<-c()

for (i in 1:7){
  if (i<6){
    rides<-rdunif(1,1000,1200)
    sim<-c(sim,rides)
  } else {
    rides<-rdunif(1,1400,1600)
    sim<-c(sim,rides)
  }
}

sim
```

```
[1] 1051 1128 1192 1111 1197 1502 1587
```

Chi Square Test

Although we have visual confirmation that ferry rides during weekdays are uniformly distributed, we would like to perform a statistical test to prove that this is in fact the case. The Chi-Square Test is generally used to compare observed frequencies with expected frequencies across categorical data. Using this comparison, we can determine if the observed data does in fact follow the hypothesis distribution. Below is the code to perform the test:

```
# Observed frequencies
observed <- c(5, 5, 5, 5, 5)

# Expected frequencies under uniform distribution
# Since we have 5 categories and assuming uniform distribution,
# each category is expected to have an equal frequency.
total_observations <- sum(observed)
num_categories <- length(observed)
expected <- rep(total_observations / num_categories, num_categories)

# Perform the Chi-Square test
```

```
chi_square_test <- chisq.test(observed, p = rep(1/num_categories, num_categories))

# Display the test results
print(chi_square_test)
```

Chi-squared test for given probabilities

```
data:  observed
X-squared = 0, df = 4, p-value = 1
```

Exercises

Question 1

SpaceX needs a countdown script for their next spaceship launch. Your task is to create a countdown that starts from 10 and goes down to 1, followed by a message saying “Liftoff!”. Create a for loop that starts at 10 and counts down to 1. After the loop ends, print “Liftoff!” to simulate the spaceship launch.

```
for (i in 10:1) {
  print(i)
}
```

```
[1] 10
[1] 9
[1] 8
[1] 7
[1] 6
[1] 5
[1] 4
[1] 3
[1] 2
[1] 1
```

```
print("Liftoff")
```

```
[1] "Liftoff"
```

Question 2

The Triangle Tavern, a popular nightlife spot, needs your help to enforce its strict age policy. The owner has observed that the youngest patrons trying to enter are around 16 years old, and the oldest are typically no more than 25. The bouncers report that 21-year-olds are the most common age group among patrons. Your task is to estimate the proportion of underage patrons that attempt to enter the bar using the triangle distribution. Create two vectors. One named `ages` that simulates the ages of 1000 customers, and a vector `entry_status` that uses a conditional statement (using the `ifelse()` function) to populate the vector with “Access Granted” for ages 21 and older, and “Access Denied” for ages under 21.

A triangular distribution is a continuous probability distribution in statistics where the possible values of a variable are defined by a minimum, maximum, and most likely value.

```
# Simulate 1000 customers using triangular distribution.
ages <- rtriang(1000, 16, 25, 21)

# Conditional function to label those 21+
entry_status<-ifelse(ages>=21,"Access Granted","Access Denied")

# Percentage of those denied access.
(round(sum(entry_status=="Access Denied")/length(entry_status),2))
```

```
[1] 0.58
```

Question 3

The Bellagio Casino in Las Vegas wants you to create a simulation of their roulette wheel, where players bet on whether the outcome will be red or black. In a standard roulette wheel, there are 18 red slots, 18 black slots, and 2 green slots (0 and 00) out of a total of 38 slots. Your task is to simulate a player betting on red for 100 spins of the roulette wheel by using a for loop and a conditional statement. If the ball lands on a red slot, the player wins and earns 1 dollar. If the ball lands on a black or green slot, the player loses 1 dollar. What is the player's total winnings or losses after 100 spins? How does the “house edge” (the presence of the green slots) affect the player's outcome over many spins?

```
#Set up wheel. rep() replicates values of x.
wheel <- c(rep('red', 18), rep('black', 18), rep('green', 2))

# Variable for simulated players winnings.
```

```
winnings <- 0

#Set seed.
set.seed(10)

#Simulated bets.
for (i in 1:100) {
  spin <- sample(wheel, 1)
  if (spin == 'red') {
    winnings <- winnings + 1
  } else {
    winnings <- winnings - 1
  }
}

winnings
```

```
[1] -14
```

Question 4

Franklin and Templeton want you to generate 4 simulated returns for the SPY (an ETF that tracks the S&P 500). They provide you with the data and histogram shown below. What distribution would you use to simulate the data? What parameters would you use?

```
library(tidyverse)
```

```
-- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
v dplyr      1.1.4      v readr      2.1.5
v forcats    1.0.0      v stringr    1.5.1
v ggplot2    3.5.1      v tibble     3.2.1
v lubridate  1.9.3      v tidyr      1.3.1
v purrr      1.0.2
```

```
-- Conflicts ----- tidyverse_conflicts() --
```

```
x dplyr::filter() masks stats::filter()
```

```
x dplyr::lag()     masks stats::lag()
```

```
x purrr::rdunif() masks extraDistr::rdunif()
```

```
i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become
```

```
data<-read_csv("https://jagelves.github.io/Data/returns.csv")
```

Rows: 1182 Columns: 3

-- Column specification -----

Delimiter: ","

chr (2): date, Stock

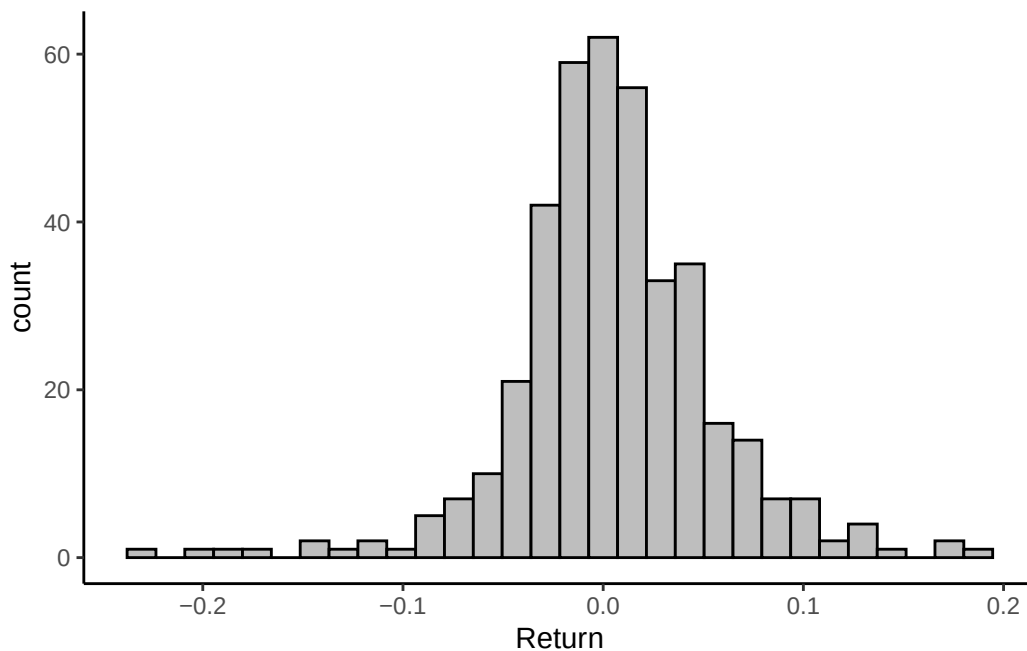
dbl (1): Return

i Use `spec()` to retrieve the full column specification for this data.

i Specify the column types or set `show\_col\_types = FALSE` to quiet this message.

```
data %>% filter(Stock=="TSLA") %>% ggplot() +  
  geom_histogram(aes(x=Return), bg="grey",col="black") +  
  theme_classic()
```

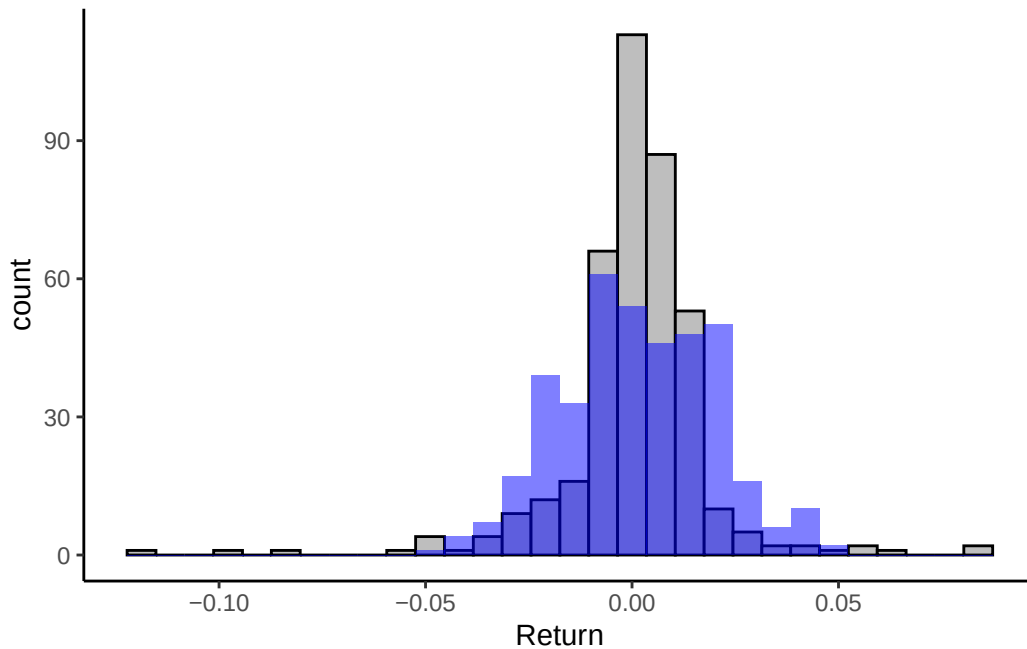
`stat\_bin()` using `bins = 30`. Pick better value with `binwidth`.



```
# Distribution looks normal. Would use mean and standard deviation as inputs for the rnorm()  
data %>% filter(Stock=="SPY") %>% ggplot() +  
  geom_histogram(aes(x=Return), bg="grey",col="black") +
```

```
geom_histogram(aes(x=rnorm(length(Return),mean(Return),
                      sd(Return))), bg="blue",
               alpha=0.5) +
theme_classic()
```

`stat\_bin()` using `bins = 30`. Pick better value with `binwidth`.
 `stat\_bin()` using `bins = 30`. Pick better value with `binwidth`.



```
#Use the following function to retrieve the mean and SD:
data %>% group_by(Stock) %>%
  summarise(Average=mean(Return),
            SD=sd(Return))
```

```
# A tibble: 3 x 3
  Stock Average    SD
  <chr>   <dbl> <dbl>
1 AAPL  0.00173 0.0257
2 SPY   0.000828 0.0177
3 TSLA  0.00511 0.0502
```

```
#Use above outputs to simulate 4 returns.  
rnorm(4,0.00173,0.025667)
```

```
[1] 0.025137694 -0.053666308 0.034830434 -0.009258298
```

Question 5

XYZ Call Center, a popular customer service provider, wants you to simulate the daily number of calls their agents receive. The company has provided you with each employee's call volume for a day and the histogram for the data. Analyze the data to determine the appropriate distribution that can be used to model the number of daily calls. You can determine the parameters for the distribution based on the historical data. Generate four simulated values representing the possible daily call volumes using the chosen distribution and parameters. What distribution would you use to simulate the daily call volumes for XYZ Call Center? What are the parameters of the chosen distribution? What are the four simulated daily call volumes?

```
library(tidyverse)  
data<-read_csv("https://jagelves.github.io/Data/CallCenter.csv")
```

```
Rows: 1100 Columns: 2
```

```
-- Column specification -----
```

```
Delimiter: ","
```

```
dbl (2): id, Calls
```

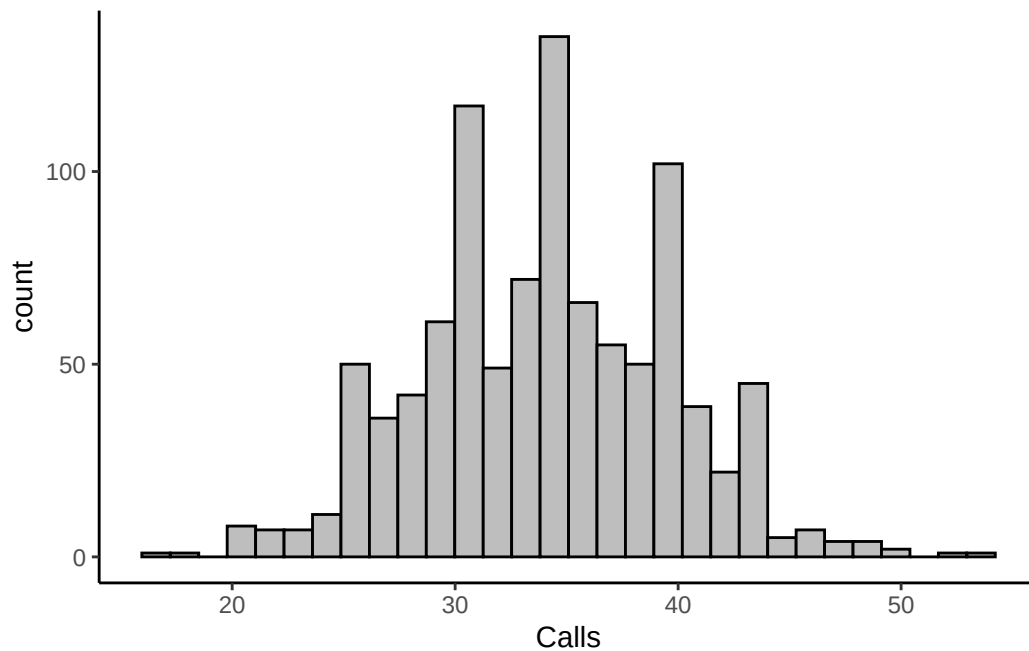
```
i Use `spec()` to retrieve the full column specification for this data.
```

```
i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```
data %>% ggplot() +  
  geom_histogram(aes(x=Calls), bg="grey",col="black") +  
  theme_classic()
```

```
`stat_bin()` using `bins = 30`. Pick better value with `binwidth`.
```

```
Warning: Removed 100 rows containing non-finite outside the scale range  
(`stat_bin()`).
```

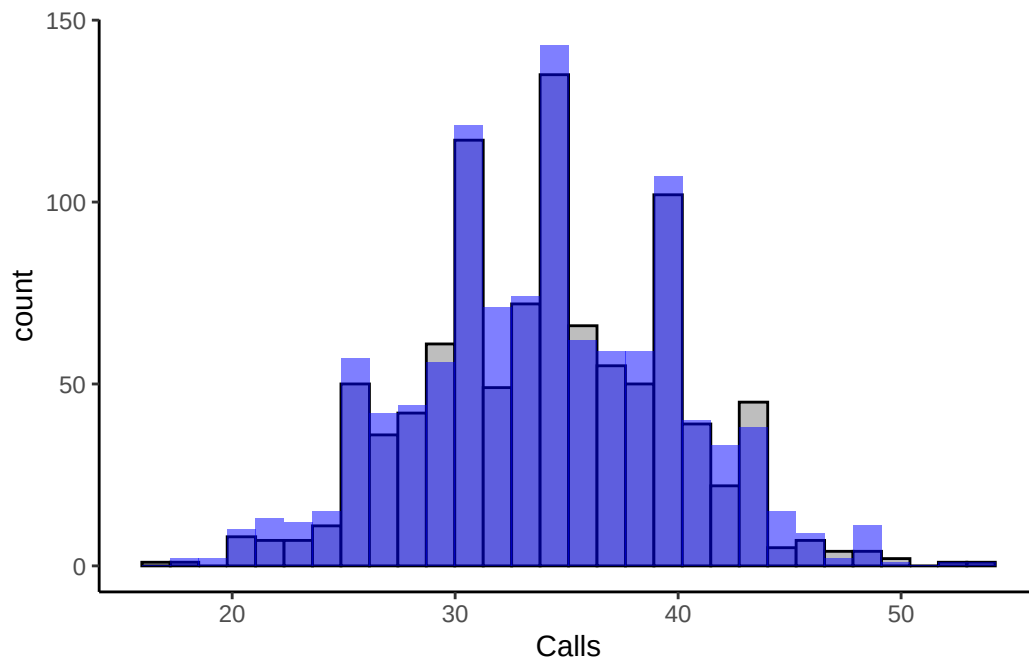


```
#Given the problem, we would use a Poisson distribution, ie distribution used to show how many
data %>% ggplot() +
  geom_histogram(aes(x=Calls), bg="grey",col="black") +
  geom_histogram(aes(x=rpois(length(Calls),mean(Calls,na.rm=TRUE))),
    bg="blue",
    alpha=0.5) +
  theme_classic()
```

`stat\_bin()` using `bins = 30`. Pick better value with `binwidth`.

Warning: Removed 100 rows containing non-finite outside the scale range
(`stat\_bin()`).

`stat\_bin()` using `bins = 30`. Pick better value with `binwidth`.



```
#Verify with chi square test
observed <- table(data$Calls)

# Calculate the mean (lambda) for the Poisson distribution
lambda <- mean(data$Calls, na.rm=TRUE)

# Calculate the expected frequencies for each observed count
expected <- dpois(as.numeric(names(observed)), lambda) * sum(observed)

# Perform the Chi-Square test
chi_square_test <- chisq.test(x = observed, p = expected / sum(expected))
```

Warning in chisq.test(x = observed, p = expected/sum(expected)): Chi-squared approximation may be incorrect

```
# Display the test results
print(chi_square_test)
```

Chi-squared test for given probabilities

data: observed

X-squared = 35.136, df = 34, p-value = 0.4141

```
#Created for simulated call days  
rpois(4,mean(data$Calls,na.rm=TRUE))
```

[1] 35 31 30 36